Date 13 / 10 / 26 Lec - 27

Class and object.

	Procedural approach	OOp approach
1.	line by line execution	Organised into classes and object
2.	we can use only built in data types list, tuple, set, dict, int, float, bool, comp, str	we can create our own custom datatype
3.	use for small script	used for large application.
4.	1000 lines of code	Half the code - 500 lines

~ Everything in python is a object

Object

Eg. num = [10, 20, 30]

list, tuple, dict, int, float, bool, comp, str :- class

Date __/__/____

create Object of class

Syntax:

Obj_name = Class_name()

OOp has 6 principles:

1. Class & Object
2. Encapsulation
3. polymorphism
4. Inheritance
5. Abstraction
6. Magic Method

1] class & Object -

Class - class is a set of rules that define how its object will behave - Blueprint

Object - object is an entity that follows class rules.

- Instance of class

class	Object
1. Blueprint	Instance.
2. Attributes + Methods	Follow rules of a class
Eg. car →	Tata, Honda
color →	Black, White, Red
animal →	Cat, dog

Date ___/___/_____

Syntax:

```
class class-name :          } class
    def fun-name():
Obj_name = class_name()  - object of class
Obj-name.fun-name()
```

class has 2 components

Attribute	Methods
Data / properties	behaviours / function

① car class -

color, milage, speed, price — avg-speed(), milage()

② Atm class -

pincode, acc balance — withdraw(), deposite()

③ Student class -

name, address, rollno, percentage — display attendance(), display report_card()

Eg.

```
class Vehicle :
    def __init__(self):
        self.color = "red"
        self.top_speed = 150
        self.milage = 15
    def display_info(self):
        print("milage is", self.milage)
        print("top_speed is", self.top_speed)
```

Page No.

Date __/__/____



```
tata = vehical()
print(tata.color)
print(tata.top-speed)
print (tata.milage)
tata.display_info()

# Easy way:
    class vehical:
    def __init__(self, color, top_speed_milage):
        self.color = color,
        self.top-speed = top-speed,
        self.milage = milage
    def display_info(self):
        print("color is", self.color)
        print("top-speed is", self.top-speed)
        print("milage is", self.milage)

honda = vehical("blacle", 150, 20)
honda.display-info()
```